

DATA STRUCTURES AND ALGORITHMS (DSA)

Objective: Understand what Big-O notation is, how it represents worst-case performance, and the basic rules for determining Big-O complexity of an algorithm

What are Data Structures and algorithms?

Data Structures manage how data is stored and accessed.

Algorithms focus on processing this data.

Why learn DSA?

- This is the foundation for almost every software eg: GPS system, Search engines, Databases.
- To decide which data structure or algorithm is best for a given situation.
- To make programs that run faster or use less memory.
- To understand how to approach complex problems and solve them in a systematic way.

Algorithmic Notation

What is an Algorithm?

An algorithm is a step-by-step set of instructions designed to perform a specific task or solve a problem. For example, a recipe for baking a cake is an algorithm. In programming, a function that sorts numbers in an array is a good example of an algorithm. Essentially, algorithms take in a particular input and they work to produce a desired output.

Why Do We Need to Measure Algorithm Performance?

When solving problems with computers, some algorithms are faster or use less memory than others. Understanding how efficient an algorithm helps us:

- Choose the best algorithm for large datasets
- Predict how long a program will take to run
- Optimize code to save time and resources

Algorithm Complexity

Introducing Big-O Notation (Worst-Case)

Big-O notation is a way to describe the upper bound of an algorithm's runtime or space usage as the input size grows. It focuses on the worst-case scenario.

Think of it like this: Imagine you're searching for a specific book in a library. Big-O tells you how long it could take in the worst possible situation – for example, if the book is the last one on the last shelf.

Big-O describes the growth rate of the algorithm's resource usage (time or space) relative to the input size (n). It tells us how the algorithm's performance scales as the input gets larger and larger.

We mainly consider the worst case scenario of the algorithm to find its time complexity in terms of Big O

Big O = “worst-case” estimate

Importance of Big O notation.

- Provides a way to describe how the runtime or space requirements of an algorithm grows as the input size increases.
- Allows programmers to compare different algorithms and choose the most efficient one for a specific problem.
- Helps in understanding the scalability of algorithms and predicting how they will perform as the input size grows.
- Enables developers to optimize code and improve overall performance.

Let's us look at some examples:

Example 1: Big-O is relative to the input size (Linear Time)

`colors = ['red', 'white', 'blue', 'black', 'green', 'yellow', 'grey']` An array of size n

`for color in colors:` Loop will run n times to print every in the array

`print(color)`

Above we have a `color-printing` algorithm with a runtime complexity of ` $O(n)$ ` (The Big-O notation). If the array had two elements, the loop will run two times; if it had 10 elements,

the loop would run 10 times; if we encode the size of the array as `n` then it will `n` times. Therefore, the big-O notation denotes that the algorithm's runtime increases linearly with the size of the input, and it is denoted as a function on `n` the input size.

Example 2: Big-O focused on worst-case scenario

```
colors = ['red', 'white', 'blue', 'black', 'green', 'yellow', 'grey']
```

```
for color in colors:
```

```
    if color == 'red':
```

```
        print('color found')
```

```
        break
```

Now, in our modified version of the `color-printing` algorithm, we know that when the loop finds `red` in the array, it prints and then breaks the loop. Because `red` is the first index of our array, the loop will only run once. However, the algorithm still has a runtime complexity of `O(n)` as we are focused on the **worst-case scenario** which occurs if `red` is the last element in the array. While, we know our algorithm will have one operation in this scenario, our big-o notation is concerned with the worst-case scenario if we gave it random arrays of colors as input.

It is worth noting that the input is not always an array, below we can see instances where the **length of array** and **size of integer** are the input sizes of an algorithm:

Example 3: String as an input

```
name = `zindua` # String of length n
```

```
count = 0
```

```
for char in name: # Runs n times
```

```
    count += 1 # Increases the count for every letter
```

```
print(count)
```

Example 4: Integer as an input

```
num = 5 # Integer of size n

sum_ = 0

for i in range(num): # Runs n times
    sum_ += i

print(sum_)
```

Both algorithms above (example 3 and 4), have runtime complexities of $O(n)$ but the type of input is different i.e. a string in the first example and an integer in the second example. While the inputs are different, the Big-O notation tells us the expected number of operations relative to the 'input size'.

Let's look at more examples that are not

Example 5: Constant Time

```
name = 'Zindua School'

print(name)
```

In the example above, the algorithm has only one operation regardless of the size of the input. Even if 'name' had 1000 characters, the 'print' statement is only one operation. Therefore, the algorithm has $O(1)$ or constant time as its runtime complexity. Even if we had 3 'print' statements, we'll still denote constant time as $O(1)$.

Example 6: Quadratic/n-squared Time

Multiplication tables of a number

```
num = 5
```

```

for i in range(1, num+1): ## Runs n times
    for j in range(1, num+1): ## Runs n times each iteration
        result = i * j
    print(f"{i} times {j} is equal to {result}")

```

The example above has $O(n^2)$ as its runtime complexity (n -squared or $quadratic\ time$). You can see that the outer loop will run n times and the inner loop will run n times for each iteration of the outer loop, thus, we'll have $n * n$ operations.

Rules behind Big-O notation

In the previous section, we have seen an ood number of algorithms and we've encoded their Big-O notations. Now, let's look at the rules to follow when determining the Big-O notation of an algorithm.

Example 7: Ignore the co-efficient

```

name = `zindua` # String of length n
count1 = 0
count2 = 0

```

Count characters in the string

```

for char in name: # Runs n times
    count += 1

```

Confirm count of characters

```

for char in name: # Runs n times
    count += 1

```

```

print(count1 == count2)

```

The algorithm above has $2n$ operations in total. The first loop will run n times and the second one will run n times as well. However, the runtime complexity will be denoted as $O(n)$ or 'linear time' as we usually ignore the coefficient, the 2 in this case. This is the same rule that applies to **example 5** where we stated that even if we had 2,3, or 10 'print' statements; so long as the operations are constant and do not change with an increase in n the input size, we'll still have the runtime complexity as $O(1)$ or 'constant time'.

Example 8: Focus on the largest term

```
num = 5
```

```
for i in range(num): ## Runs  $n^2$  times
```

```
    for j in range(num):
```

```
        print(i, j)
```

```
for i in range(num): ## Runs  $n^2$  times
```

```
    for j in range(num):
```

```
        print(i, j)
```

```
for k in range(num): ## Runs  $n$  times
```

```
    print(i, j)
```

```
for k in range(num): ## Runs  $n$  times
```

```
    print(i, j)
```

```
for k in range(num): ## Runs  $n$  times
```

```
    print(i, j)
```

```
print(num)
```

```
print(num)
```

```
print(num)
```

```
print(num)
```

The algorithm above is pretty useless but allows us to understand another important rule, the big-O notation only focus on the largest term (we are more concerned of the runtime limit as n nears infinity). If you look at the example above, the number of operations is $2n^2 + 3n + 4$ but the big-O notation will be simplified to $O(n^2)$ since we are focused on the largest term, and of course, we still ignore the coefficient.

Summary

In this lesson, we've learned what an algorithm is, why we need to measure its performance, and how we do it with the Big-O notation. Here are the most important takeaways:

- Big-O notation helps us understand how algorithms scale with input size
- The input size could be the length of an array, length of a string, or size of an integer, etc.
- We focus on the worst-case scenarios when denoting the big-O notation
- Some examples of big-O notations are $O(n)$ or linear time, $O(1)$ or constant time, and $O(n^2)$ or quadratic time
- When denoting big-O notation we ignore the coefficients and focus on the largest term; therefore, an algorithm with $4n^2 + 5n + 15$ operations will have $O(n^2)$ as its runtime complexity

Practice Questions

1. What is the Big-O runtime of the following code?

Example 9:

```
def sum_pairs(lst):  
    total = 0  
    for i in lst:  
        for j in lst:  
            total += i + j  
    return total
```

3. What is the Big-O runtime of the search algorithm below? How many operations will we have in the best-case scenario?

Example 10:

```
def search_array(arr,target):  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i  
    return -1
```

5. Which is more efficient for large inputs: $O(n)$ or $O(n^2)$? Why?